
Modelo de Programação Linear Estocástica

Planejamento Macroscópico — Ball Corporation

1. Minimização de custos • 2. Conjuntos • 3. Parâmetros • 4. Variáveis • 5. A Física do Modelo • 6. Função Objetivo • 7. Restrições • 8. Formulacção Compacta • 9. Implementação
-

Modelo de minimização de custos

O modelo busca **minimizar o custo total de operação** da malha industrial da Ball Corporation. Além de contemplar os custos de produção, estocagem e transferência, o modelo incorpora penalidades de risco: a multa por quebra de Nível de Serviço (venda perdida) e a penalidade por operação abaixo do Estoque de Segurança. O modelo é estritamente contínuo e cobre um horizonte de 15 dias.

$$\begin{aligned} \min Z = & \underbrace{\sum_{p,l,i,t} k_{i,p,l} \cdot x_{i,p,l,t}}_{\text{produção}} + \underbrace{\sum_{p,l,i,t} v_{i,p,l} \cdot e_{i,p,l,t}}_{\text{estoque}} + \underbrace{\sum_{i,o,d,t} ct_{o,d} \cdot \frac{f_{i,o,d,t}}{200.000}}_{\text{logística}} \\ & + \underbrace{\sum_{p,l,i,t} M \cdot w_{i,p,l,t}}_{\text{venda perdida}} + \underbrace{\sum_{p,l,i,t} c_{ss} \cdot falta_ss_{i,p,l,t}}_{\text{risco de estoque}} \end{aligned}$$

Sujeito às restrições físicas e operacionais R1–R10 detalhadas na Seção 7.

Conjuntos (Índices)

Símbolo	Descrição	Índice	Cardinalidade
I	SKUs (produtos finais)	i	30
P	Plantas (fábricas)	p, o, d	10
L_p	Linhas da planta p	l	3 por planta
T	Períodos $(t_1, \dots, t_{15})$	t	15 dias
$\mathcal{C}$	Triplas capáveis (i, p, l) : formato de $i = \text{for-}$ mato de $l \in L_p$	(i, p, l)	$\subseteq I \times L$

Parâmetros e Fatores de Risco

As bases estruturais (custos, demandas $\eta_{i,p,l,t}$, limites $e_{p,l}^{\max}$ e taxas de produção de placa $r_{p,l,t}$) mantêm-se conforme a extração dos arquivos CSV. Para aproximar a matemática do chão de fábrica, introduzimos:

OEE : Fator de Eficiência Global do Equipamento (Assumido: 85%).

$DM_{i,p,l}$: Demanda média diária do SKU i na linha l durante o horizonte de 15 dias.

$Dias_{ss}$: Meta de cobertura para Estoque de Segurança (Assumido: 1 dia de DM).

Variáveis de Decisão

São oito variáveis de decisão, todas contínuas e não-negativas:

Variável	Descrição	Domínio
$x_{i,p,l,t}$	Volume produzido do SKU i na linha $l \in L_p$, no dia t .	$\mathbb{R}_+$
$e_{i,p,l,t}$	Nível de estoque do SKU i na linha $l \in L_p$ ao final do dia t .	$\mathbb{R}_+$
$a_{i,p,l,t}$	Volume de atendimento local do pedido do SKU i no dia t .	$\mathbb{R}_+$
$f_{i,o,d,t}$	Volume transferido logisticamente da planta o para a planta d .	$\mathbb{R}_+$
$g_{i,p,l,t}$	Volume recebido internamente pela empilhadeira da doca para o armazém.	$\mathbb{R}_+$
$s_{i,p,l,t}$	Volume expedido internamente do armazém para a doca de carregamento.	$\mathbb{R}_+$
$w_{i,p,l,t}$	Volume de venda perdida (demanda não atendida).	$\mathbb{R}_+$
$falta_ss_{i,p,l,t}$	Volume abaixo da meta de Estoque de Segurança.	$\mathbb{R}_+$

A Física do Modelo: Modelando o Mundo Real

Modelos de Programação Linear puros sofrem do que chamamos de *A Falácia da Média*. O algoritmo otimizador tende a ser ingenuamente otimista: ele enxerga o tempo como blocos perfeitos e as máquinas como rochas inquebráveis. Para garantir validade tática, inserimos duas restrições avançadas:

1. A Ilusão da Máquina Perfeita (O Fator OEE na restrição R4)

Se a taxa manual de uma máquina é de 1.000 latas/hora, um solver clássico assumirá que entregará 24.000 latas amanhã. Na vida real, a fábrica enfrenta a troca de turno dos operadores, limpezas, trocas de bobina e falhas elétricas. Ao multiplicar a taxa máxima por um $OEE = 0.85$ na restrição R4, forçamos a matemática a prever que a máquina perderá 15% do dia. Se não ensinássemos isso ao solver, ele venderia uma capacidade fantasma, colidindo com a Teoria das Filas no chão de fábrica.

2. O Medo do Imprevisto (Estoque de Segurança como Soft Constraint)

A equação física R2 decreta que a fábrica inicie com os galpões inteiramente vazios ($e^0 = 0$). Queremos que o algoritmo atue como um gerente de fábrica prudente, que constrói um Estoque de Segurança ao longo do tempo para se blindar da variância de pedidos.

No entanto, se impusermos uma **Restrição Rígida** (Hard Constraint) como $e \geq DM$, o modelo irá colapsar (*Infeasible*) no dia $t = 1$, pois é fisicamente impossível nascer com latas no estoque inicial zero.

A solução (Restrição R10): Injetamos a variável de risco falta_ss.

$$e_{i,p,l,t} + \text{falta_ss}_{i,p,l,t} \geq \text{Meta de Segurança}$$

Isso dá empatia ao modelo: se ele não tiver estoque, a equação não quebra, mas a variável falta_ss assume o risco e paga uma pequena multa.

Função Objetivo

$$\begin{aligned} \min Z = & \sum_{p \in P} \sum_{\substack{l \in L_p \\ (i,p,l) \in C}} \sum_{i \in I} \sum_{t \in T} k_{i,p,l} \cdot x_{i,p,l,t} + \sum_{p \in P} \sum_{l \in L_p} \sum_{i \in I} \sum_{t \in T} v_{i,p,l} \cdot e_{i,p,l,t} \\ & + \sum_{i \in I} \sum_{o \in P} \sum_{\substack{d \in P \\ d \neq o}} \sum_{t \in T} ct_{o,d} \cdot \frac{f_{i,o,d,t}}{200.000} + \sum_{p,l,i,t} 1.000.000 \cdot w_{i,p,l,t} + \sum_{p,l,i,t} 5 \cdot \text{falta_ss}_{i,p,l,t} \end{aligned}$$

O Comportamento do Otimizador: Ao precificarmos o cancelamento do pedido (w) em R\$ 1.000.000 e a violação do estoque de segurança (falta_ss) em meros R\$ 5, ensinamos a inteligência algorítmica a ter **empatia comercial**: O cliente sempre vem primeiro, mas, havendo folga fabril, encha o armazém para não pagar a taxa de risco.

Restrições Físicas

R1 — Limite de armazenagem.

O espaço físico do armazém é compartilhado entre todos os SKUs:

$$\sum_{i \in I} e_{i,p,l,t} \leq e_{p,l}^{\max} \quad \forall p \in P, l \in L_p, t \in T$$

R2 — Balanço de estoque.

Conservação de massa do armazém acoplada no tempo ($e_{i,p,l,0} = 0$):

$$e_{i,p,l,t} = e_{i,p,l,t-1} + x_{i,p,l,t} + g_{i,p,l,t} - s_{i,p,l,t} - a_{i,p,l,t} \quad \forall i, p, l, t \in T$$

R3 — Atendimento à demanda.

O atendimento local a somado à venda perdida w iguala a demanda η :

$$a_{i,p,l,t} + w_{i,p,l,t} = \eta_{i,p,l,t} \quad \forall i, p, l, t \in T$$

R4 — Capacidade produtiva (Ajustada com OEE).

A soma das horas consumidas por todos os SKUs na linha l sofre a ação da ineficiência mecânica global antes de bater no teto do turno de 24 horas:

$$\sum_{i \in I} \frac{x_{i,p,l,t}}{r_{p,l,t} \cdot \text{OEE}} \leq 24 \quad \forall p \in P, l \in L_p, t \in T$$

R5 — Capacidade logística de expedição.

Cada planta expede no máximo 80 caminhões por dia (saída):

$$\sum_{i \in I} \sum_{\substack{d \in P \\ d \neq p}} \frac{f_{i,p,d,t}}{200.000} \leq 80 \quad \forall p \in P, t \in T$$

R6 e R7 — Nó de transbordo da fábrica (docas).

O frete não entra magicamente no estoque; ele é balizado pelas docas (g e s):

$$\sum_{l \in L_p} g_{i,p,l,t} = \sum_{\substack{o \in P \\ o \neq p}} f_{i,o,p,t} \quad (\text{R6})$$

$$\sum_{l \in L_p} s_{i,p,l,t} = \sum_{\substack{d \in P \\ d \neq p}} f_{i,p,d,t} \quad (\text{R7})$$

R8 — Capabilidade de formato e R9 — Não-negatividade.

$$x_{i,p,l,t} = 0 \quad \forall (i, p, l) \notin \mathcal{C}, t \in T \quad (\text{R8})$$

Todas as 8 matrizes de variáveis adotam $Var \geq 0$ (R9).

R10 — Soft Constraint de Estoque de Segurança.

O estoque de final de dia mais a complacência do risco são cobrados:

$$e_{i,p,l,t} + falta_ss_{i,p,l,t} \geq DM_{i,p,l} \cdot Dias_{ss} \quad \forall (i, p, l) \in \mathcal{C}, t \in T$$

Formulação Compacta do PL

min

$$Z = \sum_{p,l,i,t} k_{i,p,l} \cdot x_{i,p,l,t} + \sum_{p,l,i,t} v_{i,p,l} \cdot e_{i,p,l,t} + \sum_{i,o,d,t} ct_{o,d} \cdot \frac{f_{i,o,d,t}}{200.000} \\ + \sum_{p,l,i,t} M \cdot w_{i,p,l,t} + \sum_{p,l,i,t} c_{ss} \cdot falta_ss_{i,p,l,t}$$

s.a.

$$\sum_i e_{i,p,l,t} \leq e_{p,l}^{\max} \quad \forall p, l, t \quad (R1)$$

$$e_{i,p,l,t} = e_{i,p,l,t-1} + x_{i,p,l,t} + g_{i,p,l,t} - s_{i,p,l,t} - a_{i,p,l,t} \quad \forall i, p, l, t \quad (R2)$$

$$a_{i,p,l,t} + w_{i,p,l,t} = \eta_{i,p,l,t} \quad \forall i, p, l, t \quad (R3)$$

$$\sum_i \frac{x_{i,p,l,t}}{r_{p,l,t} \cdot OEE} \leq 24 \quad \forall p, l, t \quad (R4)$$

$$\sum_{i,d \neq p} \frac{f_{i,p,d,t}}{200.000} \leq 80 \quad \forall p, t \quad (R5)$$

$$\sum_{l \in L_p} g_{i,p,l,t} = \sum_{o \neq p} f_{i,o,p,t} \quad \forall i, p, t \quad (R6)$$

$$\sum_{l \in L_p} s_{i,p,l,t} = \sum_{d \neq p} f_{i,p,d,t} \quad \forall i, p, t \quad (R7)$$

$$x_{i,p,l,t} = 0 \quad \forall (i, p, l) \notin \mathcal{C}, t \quad (R8)$$

$$e_{i,p,l,t} + falta_ss_{i,p,l,t} \geq DM_{i,p,l} \cdot Dias_{ss} \quad \forall (i, p, l) \in \mathcal{C}, t \quad (R10)$$

$$x, e, a, f, g, s, w, falta_ss \geq 0 \quad \forall i, p, l, o, d, t \quad (R9)$$

Aproximadamente **139.500** variáveis contínuas no total.

Implementação em Julia/JuMP

O código abaixo mapeia a lógica estocástica. Note o uso de `mapa_tempo` para blindar o solver contra saltos cronológicos de final de mês, garantindo o balanço perfeito de R2.

```

using JuMP, Gurobi, CSV, DataFrames, Printf, Dates

# ===== PAINEL =====
fator_oe = 0.85 # <--- Meta de Eficiencia (Gargalo f'isico)
dias_seguranca = 1.0 # <--- Meta de Cobertura de Estoque

const GRB_ENV = Gurobi.Env()

function resolver_cenario(cenario, df_log)
    caminho = "C:\\Dados\\ocupacao_" * cenario * "\\"
    df_need = CSV.read(caminho * "input_production_need_artificial.csv", DataFrame)
    df_rate = CSV.read(caminho * "input_production_rate_artificial.csv", DataFrame)
    df_costs = CSV.read(caminho * "input_sku_costs_artificial.csv", DataFrame)
    df_limit = CSV.read(caminho * "input_line_storage_limit_artificial.csv", DataFrame)

    # ---> MAPEAMENTO CRONOLOGICO (Previne erros em viradas de mes)
    datas_demanda = [split(string(d), " ")[1] for d in df_need.deadline if !ismissing(d)]
    datas_taxa = [split(string(d), " ")[1] for d in df_rate.ref_date if !ismissing(d)]
    datas_unicas = sort(unique(vcat(datas_demanda, datas_taxa)))
    mapa_tempo = Dict{data => t for (t, data) in enumerate(datas_unicas)}

    function get_t(data_str)
        d_limpa = split(string(data_str), " ")[1]
        return get(mapa_tempo, d_limpa, -1)
    end

    df_need.prod_need = parse_br_float.(df_need.prod_need)

    I = unique(df_need.product_id)
    P = unique(df_rate.plant)
    T = 1:length(datas_unicas)
    L_p = Dict{p => unique(df_rate[df_rate.plant .== p, :production_line]) for p in P}

    k = Dict{(r.product_id, r.plant, r.production_line) => parse_br_float(r.
        production_cost) for r in eachrow(df_costs)}
    v = Dict{(r.product_id, r.plant, r.production_line) => parse_br_float(r.inventory_cost
        ) for r in eachrow(df_costs)}
    ct = Dict{(r.origem, r.destino) => parse_br_float(r.custo_total_frete) for r in
        eachrow(df_log)}
    e_max = Dict{(r.plant, r.production_line) => parse_br_float(r.storage_limit) for r in
        eachrow(df_limit)}

    eta = Dict{(i,p,l,t) => 0.0 for i in I for p in P for l in L_p[p] for t in T}
    for r in eachrow(df_need)
        t = get_t(r.deadline)
        chave = (r.product_id, r.plant, r.production_line, t)
        haskey(eta, chave) && (eta[chave] += r.prod_need)
    end

    r_taxa = Dict{(p,l,t) => 0.0 for p in P for l in L_p[p] for t in T}
    for r in eachrow(df_rate)
        t = get_t(r.ref_date)
        r_taxa[(r.plant, r.production_line, t)] = parse_br_float(r.rate)
    end

    format_linha = Dict{r.production_line => strip(lowercase(string(r.sku_size_shape)))
        for r in eachrow(df_rate)}
    format_sku = Dict{r.product_id => strip(lowercase(string(r.sku_size_shape))) for r
        in eachrow(df_need)}

    C = Set{(i, p, l) for i in I, p in P, l in L_p[p]
        if get(format_sku, i, "none") == get(format_linha, l, "none_L")}

    # DM (Demanda Media Diaria)
    demanda_media = Dict{(i,p,l) => sum(get(eta, (i,p,l,t), 0.0) for t in T) / 15.0 for i
        in I for p in P for l in L_p[p]}

    modelo = Model() -> Gurobi.Optimizer(GRB_ENV)
    set_silent(modelo)

    @variable(modelo, x[i in I, p in P, l in L_p[p], t in T] >= 0)
    @variable(modelo, e[i in I, p in P, l in L_p[p], t in T] >= 0)
    @variable(modelo, a[i in I, p in P, l in L_p[p], t in T] >= 0)
    @variable(modelo, f[i in I, o in P, d in P, t in T; o != d] >= 0)
    @variable(modelo, g[i in I, p in P, l in L_p[p], t in T] >= 0)
    @variable(modelo, s[i in I, p in P, l in L_p[p], t in T] >= 0)
    @variable(modelo, w[i in I, p in P, l in L_p[p], t in T] >= 0)
    @variable(modelo, falta_ss[i in I, p in P, l in L_p[p], t in T] >= 0) # <--- NOVO (R10)

```